Executable Archaeology of Programs That Model Programming: Reanimating the Heuristic Compiler (1961) and KBEmacs (1984)

Jeff Shrager[1], Kent M. Pitman[2], and Mia Golec[3]

[1]https://jshrager.org
[2]https://github.com/netsettler
[3]Carnegie Mellon University, Pittsburgh, PA, USA

Abstract

Early AI programs were often also theories of how people think, and programs that wrote programs were, in part, theories of how people program. Here we report on the reanimation of two such programs that offer two different theories of programming and were developed for quite different purposes. The first, Herbert Simon's Heuristic Compiler (1961), was an experimental model of programming as problem-solving, applying the means-ends analysis methods of the General Problem Solver to investigate the nature of the programming task. The second, KBEmacs (1984), was a working prototype of an intelligent programming assistant developed by MIT's Programmer's Apprentice project, built on the view that expert programmers recognize and combine familiar patterns, or "cliches." It let a programmer build up programs by combining cliches through a stylized English command language. We approach both through the methodology of Executable Archaeology, the study of historical computational systems by running their original code, a process we call "reanimation." Our reanimation of the Heuristic Compiler reproduces the compiled code appearing in Simon's 1963 paper, and our reanimation of KBEmacs reproduces the SQRT sequence of its December 1984 self-test. In order to highlight the reflexivity of this project, we tried to have most of the detailed effort in these reanimations carried out by a modern AI-assisted programming tool, Anthropic's Claude Code (Opus 5.5), although it started from a significant base of existing software. The explicit, executable theories of programming that both systems embodied are largely absent from today's far more capable AI coding assistants.

Keywords: Executable Archaeology, History of AI, Automatic Programming, Heuristic Compiler, KBEmacs, Programmer's Apprentice, Programming Cliches, Psychology of Programming, Cognitive Models of Programming, Claude Code

1 Introduction

Modern programmers increasingly work alongside AI-based tools built on large language models (LLMs) and trained on enormous corpora of source code and other programming-related materials. Although the scale and fluency of the modern tools is truly novel, the aspiration of a program that takes part in the work of programming is nearly as old as programming itself. In the 1950s what we now call "compilation" was referred to as "automatic programming" [1, 21]. And within a few years of the founding of artificial intelligence, researchers were asking whether the methods of AI could be turned on the task of programming itself [2].

Programming has also long been studied as a human activity. Weinberg's *The Psychology of Computer Programming* [34] treated programming as a behavior to be observed, and by the early 1980s Soloway and others theorized that expert programmers usually work from stereotyped "programming plans" – reusable code schemas – rather than from first principles [32]. Similarly, most early AI was intended either as formal theories of human cognition (e.g., [15, 14]), or depended upon intuitive theories of human cognition. In that framing, a program that writes programs, or helps a programmer do so, is at least an intuitive theory of programming, but one that could also be helpful to a human programmer.

This paper describes the reanimation of two historical systems that offer two such theories, though they were built for different ends. The first is Herbert A. Simon's *Heuristic Compiler*, written in IPL-V in 1961 and described in a RAND report that year [28] and in the *Journal of the ACM* in 1963 [30]. Simon treated programming as a 'problem' in the sense in which that term was used in GPS, the General Problem Solver of Newell, Shaw, and Simon [13]: Given a description of what a routine should do, Simon's Heuristic Compiler searched for differences between the routine as it stands and the desired routine, and applied operators to reduce those differences.

The second system is *KBEmacs*, the Knowledge-Based Editor (in Emacs). Built at MIT's Artificial Intelligence Laboratory, KBEmacs was the culminating demonstration of the Programmer's Apprentice project [37, 39, 20]. The user of KBEmacs guided the computer by describing the desired program using stylized English and mentioning specific "cliches". KBEmacs looked up the proto-code underlying the indicated cliches, supplied the details, and wrote the code by combining the cliche proto-code into the program under construction. This process was quite similar to how psychologists studying programming, such as Soloway (e.g., [32]) theorized human programmers reasoned (although Soloway used the term "programming plan" instead of "cliche").

Although the Heuristic Compiler and KBEmacs were separated by a quarter of a century, by programming language and platform (IPL-V on 1960s mainframes v. Zetalisp on MIT Lisp machines and their descendants), and by research tradition (the study of human cognition v. applied AI), they stake out a great deal of the conceptual territory that today's AI coding tools occupy: How a programming task is specified, where the knowledge and skill needed to carry it out come from and where it resides, how work is divided between human and machine, and how the system checks what it has done.

We approach the study of these artifacts as exercises in *Executable Archaeology* employing that field's core methodology, "reanimation": studying historical programs by running them on emulation stacks from code that is as close as possible to the original [10, 12]. The Heuristic Compiler was transcribed from a 1961 line-printer listing recovered from Simon's papers at CMU, and run on the Common Lisp IPL-V interpreter built earlier to run the Logic Theorist [26]. KBEmacs was transcribed from a four-volume January 1985 printout and run under a Zetalisp compatibility layer on top of SBCL, a modern Common Lisp. Both reanimations were themselves carried out largely by an AI coding assistant: Anthropic's Claude [5], working in its command-line form, Claude Code. (The division of labor is described in "Author Contributions and Use of AI".)

2 Historical Background

2.1 The Heuristic Compiler

The goal of Newell, Shaw, and Simon's work at RAND and Carnegie Tech in the 1950s was to instantiate theories of human thought as computer programs – what we (and many others) call "computational models of cognition", or merely "models". Their first model, in 1955/6, was called the "Logic Theorist" (LT). LT searched for proofs of theorems in *Principia Mathematica* [16, 12].

Later, they created the "General Problem Solver" (1959) which generalized the method applied in LT into a theory of "means-ends analysis", theorizing that this is a core cognitive skill [13]. Simon's 1963 summary of means-ends analysis is worth quoting in full, because the Heuristic Compiler implements it almost literally:

> Given the present and desired objects, find a difference between them. Next, find an operator relevant to the difference; determine if the operator can be applied to the present object. If so, apply it. (If not, describe the objects to which it would apply and transform the present object into an object of that kind—a new "desired object.") Take the new object thus obtained as the present object and repeat the process. [30, p. 494]

A proof and a program are not as different as it might seem. A proof is essentially a program that outputs a theorem, given inputs that are axioms and prior proved theorems. In this sense (Simon continues), "a theorem-proving system can be regarded ... as a program-writing system. ... if we can formulate a programming goal as a difference between a present and a desired object, we can presumably use the same processes ... to generate a program" [30, p. 495]. The same observation, in a deductive rather than heuristic setting, later became the foundation of program synthesis from theorem proving [8, 11], and of program verification by theorem proving [3].

Simon's goal was *not* to build an automatic programming tool: "no claims will be made for the heuristic programming procedures described here as practical approaches to the construction of compilers. Their interest lies in what they teach us about the nature of the programming task" [30, p. 493].[1]

2.2 The Programmer's Apprentice

The Programmer's Apprentice project began at MIT in the mid-1970s, with Charles Rich and Howard Shrobe's *Initial Report on a LISP Programmer's Apprentice* [18]. These researchers were realistic about the potential capabilities of such an AI-based programming tool at that time: "Recognizing that it will be a long time before it is possible to fully duplicate an expert programmer's abilities, the project seeks to develop an intelligent assistant system ... which will help a programmer in various phases of the programming task" [39, p. 47].

Over the following decade the project developed three central ideas: *Cliches* are standard computational methods, such as a linear search, a report generator, or successive approximation, each with *roles* that vary from use to use and a fixed surrounding structure. *Plans* are a language-independent representation of programs as hierarchical flowcharts with explicit data and control flow, which Rich formalized in a Plan Calculus [19, 20]. The *assistant approach* was a division of labor wherein the programmer makes design decisions and the apprentice handles the routine details. Waters's dissertation developed the analysis of programs into plans [36]. Note how these ideas align with those of the psychology of programming at that same time, for example Soloway's programming plans [32].

KBEmacs was the realization of these ideas in a working editor, built inside the Zmacs editor of the MIT Lisp machines and their descendants. KBEmacs could create and modify moderately complex programs, within its range of available cliches – in both Lisp and Ada. Its best-known demonstration builds an Ada report generator, `UNIT-REPAIR-REPORT`, from five commands such as "Define a simple-report procedure UNIT-REPAIR-REPORT" and "Fill the enumerator with a chain-enumeration of UNITS and REPAIRS" [39, p. 48],[38].

[1] This method is called, appropriately, "task analysis" in [14]

KBEmacs was admittedly a research prototype. Waters wrote that it knew "only a few dozen cliches" where a practical tool "would need to know many hundreds," and that, its ideas having been used "for essentially all [they are] worth," KBEmacs "has been mothballed" [39, pp. 54–55]. The project continued with work on the Plan Calculus, on automated reasoning, and on requirements and design apprentices, culminating in Rich and Waters's 1990 book [22].

3 Reanimating the Heuristic Compiler

3.1 The Listing

The source used here for the Heuristic Compiler is a line-printer listing headed `HEURISTIC CODER 7/16/61`, discovered in the Herbert A. Simon Papers at the Carnegie Mellon University Archives [29]. This listing is dated shortly after the RAND report P-2349 ([28]), and is very nearly the program described in that report. The listing is 94 line-printer pages, and about 4,470 lines (cards). We call this "the deck". The listing also carries various hand markings indicating a handful of proposed corrections. The deck contains an executive and two "compilers"[2]; a natural-language front end (a character scanner, dictionary, and parser) intended to let the compiler accept English definitions; an IPL-V knowledge base; and a renderer that takes the final step, in which the program's internal representation is printed as IPL-V text.

3.2 Everything Is a Description List

The central representational idea of the Heuristic Compiler is that a *routine* is an object with attributes stored as an IPL-V description list (analogous to a lisp property list). Each attribute is one possible definition of the routine: its IPL name, its *functional description* (i.e., a process name plus arguments, such as "INSERT ( ) AT END OF ( )"), its *state description* (i.e., which cells it affects, and what each holds before and after), its flow diagram, and its IPL-V code. Within the IPL code rendered in this representation, IPL-V instructions are themselves description lists with attributes for each field. For example, a symbol, such as `J3`, is stored as a description list with a region (the letter "J") and a location (the integer 3). The program never handles IPL code as text. Its compilers build and transform these structures, and the renderer prints them out as IPL-V text only at the end.

The Heuristic Compiler's executive implements a means-ends problem-solving loop whose *differences* are kinds of definitions and whose *operators* are called "compilers". The executive finds the closest definition for the routine, looks up the relevant compiler in a table of connections, applies it, and tests whether the routine has improved. If the routine has not improved, the executive gives up. (There was no search in the AI sense, no handling of failed paths nor any backtracking.)

The *state description compiler* is the most GPS-like part of the Heuristic Compiler. It compares the input and output states of each affected cell, classifies the largest difference (addition, deletion, replacement, or copying), looks up an operator relevant to it, binds the operator's variables to the actual cells, and then recursively compiles two subproblems: getting from the original input state to the required state, and getting from the operator's output state to the desired output. Its knowledge of IPL-V consists of exactly two operators, LOAD and REPLACE, each itself a tiny routine with its own state description.[3]

[2] Simon uses "compiler" at two levels. The Heuristic Compiler as a whole is a compiler; within it, two "compilers," one for state descriptions and one for functional descriptions, serve as the operators of its means-ends loop.

[3] The state description compiler in [31] differs from the one we reanimated in representing states by bare symbols, whereas ours uses described states.

The *functional description compiler* works by analogy. Given a routine described as, say, "INSERT (1) AT END OF THE VALUE OF THE ATTRIBUTE (0) OF (2)," it searches the routines already compiled for one with the same *process*, here "INSERT (0) AT END OF (1)," whose code is the single primitive `J65`. It then adapts that routine to the new arguments, moving the inputs into working cells, compiling code to fetch each argument, recursing when an argument is itself a described computation (a "determiner"), and wrapping the result in instructions that save and restore the working cells.

3.3 Running The Heuristic Compiler

The recovered deck runs on the Common Lisp IPL-V interpreter originally written to run the Logic Theorist [26, 27]. Claude extended the interpreter with several primitives (J-functions) needed by the Heuristic Compiler but not required by the Logic Theorist, and added a start card, borrowed from [31]. With these changes, the recovered deck reproduces Simon's published examples.

Compiling from a functional description. Given the functional description of INSERT AT END OF VALUE LIST, the program produces `J13 J52 11W2 11W0 J10 11W1 J65 J32 0`, exactly matching the code in the 1963 paper [30, pp. 499–500] (Figure 1). A print apparently left in routine U113 shows the code growing one argument at a time, which fortuitously prints out the sequence of stages through which the paper walks its readers. In Simon's runs, as in ours, the functional description is supplied as a hand-coded list structure, not as typed English; the phrase is printed from it, but the compiler matches processes by symbol identity. The English front end in the deck has not yet been reanimated (see "Next Steps").

```
FIND THE VALUE-OF THE ATTRIBUTE (0) OF (1)              <- an already-compiled routine

                                     J10   0

                                     J65   0           <- the code of INSERT (0) AT END OF (1)

                                    11W1               <- fetch argument (1), now in W1
                                     J65   0

                                    11W2               <- fetch the determiner's arguments
                                    11W0                  and apply its code (J10)
                                     J10
                                    11W1
                                     J65   0

J13
INSERT (1) AT END OF  THE VALUE-OF THE ATTRIBUTE (0) OF (2)

                               J13    J52              <- preserve working cells W0-W2
                                    11W2
                                    11W0
                                     J10
                                    11W1
                                     J65
                                     J32   0           <- restore them and exit
```

Figure 1: Output from the reanimated Heuristic Compiler compiling INSERT AT END OF VALUE LIST from its functional description. The intermediate stages are printed by the program (via a print in routine U113 that appears to be a debugging aid left in the deck). The final code matches that displayed in Simon's 1963 paper. (Annotations after `<-` are ours.)

Compiling from a state description. Simon's running example of the state description compiler is J3, SET SIGNAL MINUS: a routine that replaces the top symbol of the signal cell H5 with the symbol J3 ("minus"). The state description says that H5 goes from *some symbol on top of the rest of its pushdown list* to *J3 on top of the same rest*. The compiler finds a replacement difference in a named cell; finds REPLACE relevant, with its variable bound to H5, giving `20H5`; sets up the prefix subproblem "get J3 into the communication cell," an addition, for which LOAD is relevant, giving `10J3`; finds no suffix subproblem; and assembles `10J3 20H5 0`, the code in the paper [30, p. 498].

We noted one exception in the reproduced results: Our recovered deck did not initially compile J3. The reason, we discovered, is that our listing's J3 had both a functional and a state description, but the executive always prefers the functional description on the assumption that "it is easier to compile from a functional description than from a state description" [30, p. 502]. As no already-compiled routine shares J3's process, the functional description compiler fails, the executive sees no progress and gives up without trying the state description. The published paper's J3 result is only produced by our reanimation when the deck is repaired by *removing* J3's functional description, after which the state description compiler produces exactly the published code. In the 1963 listing (in [31]) the executive is identical, but the cards that give J3 a functional description are numbered as though they were later additions, suggesting that the paper's J3 came from an earlier version in which, like our repair, J3 had only a state description.

Flow diagrams. The paper's last section handles routines with branches and loops by decomposing them into *segments* joined by a flow diagram. Our reanimation reassembles routine J77

("test whether there is a symbol equal to 0ACCUMULATOR on list 1ACCUMULATOR") from its segments, and decomposes it again into the paper's flow diagram.

4 Reanimating KBEmacs

4.1 The Listings

The KBEmacs source that we used for its reanimation is a January 1985 printout comprised of four bound volumes of Zetalisp code, preserved by Kent Pitman, a member of the KBEmacs team. The scans are available at https://github.com/netsettler/netsettlement/tree/main/kbe [17]. These cover the system as of KBE version 358.0x (December 1984), although individual files carry their own dates and versions. Volume 1 opens with the output of an extremely useful automatic self-test, run in December 1984 on one of the MIT Lisp machines or their descendants. The self-test exercises six command sequences and compares the generated code with the expected code. This specific run reports that three sequences (HASH, SQRT, and SUMLIST) succeeded, but that three (BUCKET, DELETE, and MEMQ) failed.

The 87 files were transcribed (mostly by ChatGPT [4]) into about 34,500 lines of source, available at https://github.com/jeffshrager/kbemacs [24].

4.2 Text and Plans

A KBEmacs user works in a stylized English command language phrased in terms of cliches and their roles. The cliche library is written as code with holes, as in the definition of successive approximation in Figure 2.

```
(DEFINE-CLICHE SUCCESSIVE-APPROXIMATION
   (PRIMARY-ROLES (INITIAL-APPROXIMATION)
    DESCRIBED-ROLES (INITIAL-APPROXIMATION CRITERION IMPROVEMENT-STEP)
    COMMENT "iteratively improves an approximation until {the criterion} is
             satisfied" ...)
  (LET* ((APPROXIMATION {the input initial-approximation}) )
    (LOOP DO
      (IF ({the criterion} APPROXIMATION) (RETURN APPROXIMATION) )
      (SETQ APPROXIMATION ({the improvement-step} APPROXIMATION) ))))
```

Figure 2: A cliche from KBEmacs's library (`kbelib.lisp`, 1984). The braces are KBEmacs read syntax marking *roles*, the parts that vary from use to use; the surrounding code is fixed.

Similar to the way in which the Heuristic Compiler worked with internal representation of IPL-V code, KBEmacs's cliches are not stored as text. When the library loads, each definition is run through KBEmacs's *analyzer*, which turns the cliche code into a plan: a set of representations describing segments, ports, data flow, control flow, and roles. In a plan, a `LOOP` with a `SETQ` disappears; what remains is a temporal composition of a sequence of states. The knowledge-based editor operates on plans, and after every command the *coder* turns the plan back into program text, deciding where to nest expressions and where to introduce variables, preferring names the programmer has already used. Waters called this two-way translation the *representation shift*, and argued that it gave plans locality, additivity, and modularity that text lacks [39]. Because the user may also simply edit the text, KBEmacs must keep text and plan consistent in both directions.

4.3 Running KBEmacs (headless)

KBEmacs depends heavily on the Lisp Machine environment of its time, including Flavors (the Lisp Machine's object system), the Zmacs editor, Lisp Machine processes, a special LetS loop notation, and Zetalisp itself, which is similar to, but not exactly the same as, modern Common Lisp. Emulators survive for the MIT CADR Lisp Machine (usim) and for Symbolics Open Genera, but no usable environment compatible with this KBEmacs release was available to us. Both include Flavors, but Open Genera, the closer of the two, is not freely available, and it corresponds to a system about a decade later than the one KBEmacs ran on.

The reanimation targeted the *headless* core: the database, analyzer, cliche library, command language, knowledge-based editor, and coder, leaving out the editor, windows, actors, and the Ada front end [24].

The 1985 source, with correction of minor typos and transcription errors, is loaded into SBCL (`https://www.sbcl.org`), a modern Common Lisp[4], on top of a Zetalisp compatibility layer that Claude assembled and debugged. The bulk of that layer derives from g000001's Common Lisp port of Spice Lisp Flavors [7]. The rest of the layer supplies the parts of Zetalisp and the Lisp Machine that Common Lisp lacks or does differently. A small test harness was also created to replay the self-test sequences from the 1984 code and compare the results with what was expected.

The SQRT sequence (Figure 3) builds a square-root routine from three cliches in seven commands, and then builds a variant with an extra parameter in seven more. After the first command that names a cliche, KBEmacs holds the cliche's structure with its roles still open as holes. Each further command fills a role with code or with another cliche, whose own unfilled roles appear as new holes. After seven commands the plan has 32 segments and the program is complete.

```
>Reinitialize database.
>Define a program MYSQRT with a parameter NUM.
>Implement it as a successive-approximation.
>Implement the initial-approximation as [1.] .
>Implement the improvement-step as an average of [APPROXIMATION]
   and [(// NUM APPROXIMATION)] .
>Implement the criterion as an equality-within-epsilon of
   [(* APPROXIMATION APPROXIMATION)] and [NUM] .
>Implement the epsilon as [0.0001] .

(DEFUN MYSQRT (NUM)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) 0.0001)
          (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))

The program MYSQRT is a successive-approximation. The initial-approximation is 1.
The criterion is an equality-within-epsilon ...
```

Figure 3: KBEmacs, running in 2026, replaying the first half of the SQRT sequence from its December 14, 1984 self-test. The generated code is `equal` to the code recorded in 1984. The final lines are KBEmacs's own description of the program, generated from its plan, as recorded in the 1984 self-test (abridged).

[4]Coincidentally, Spice Lisp, which was built at CMU, where Simon and Newell taught, begat CMUCL, and SBCL is a fork of CMUCL.

All fourteen commands are parsed correctly and the generated programs are `equal` (in the Lisp sense) to the 1984 generated code. These programs also actually work: `(mysqrt 2.0)` returns 1.4142157.

5 Then and Now: Two Historical Systems and Today's AI Coding Assistants

Neither the Heuristic Compiler nor KBEmacs is an ancestor of today's AI coding assistants in any direct technical sense. The historical programs are loaded with proto-programs or cliches, whereas LLMs are trained on huge volumes of example code, as well as programming-related content, such as advice garnered from online forums, such as Stack Overflow.[5] Also, the Heuristic Compiler and KBEmacs operate on explicit internal representations of the code, whereas LLMs operate on the program text and contextual materials such as the programmer's guidance prompts.[6] Nonetheless the questions these two systems were built to explore are recognizably the questions that shape AI-assisted programming today. But the comparisons that follow are conceptual continuities, not architectural equivalences.

How is a programming task specified? Simon's two compilers differ precisely in how the task is described. A state description says what the machine looks like before and after, which is close to what we would now call a specification by examples or by pre- and post-conditions, and close to the role unit tests play in directing a modern assistant. A functional description is an imperative phrase, which is close to a natural-language prompt. Simon drew a conclusion that remains apt: programmers use both, but "at the higher levels ... a language of functions appears generally the more natural and powerful," and "the mappings from functional descriptions to process language are simpler and more direct than the mappings from state descriptions" [30, p. 506]. KBEmacs chose the functional side and gave it a vocabulary of named algorithms – cliches. Today's assistants accept both kinds of description, and much of the craft of using them lies in combining functional prompts with tests that the AI can use to check its work and self-correct.

Where does the knowledge come from? Both historical systems are knowledge-starved by modern standards. The Heuristic Compiler's state description compiler knows exactly two operators; its functional description compiler can compile a routine only if a routine with the same process has already been compiled. KBEmacs knew "a few dozen" cliches; Waters estimated that a practical tool would need "many hundreds" [39, p. 55]. Both systems identified reuse of known solutions as the heart of the matter: Simon's functional compiler works by finding an analogous routine and adapting it, and the Programmer's Apprentice rests entirely on the claim that programs are built from background knowledge. That claim was also a psychological one: it is essentially what Soloway and his colleagues were finding in human programmers, whose expertise lay in a store of "programming plans" [32]. What neither could solve was acquiring that knowledge at scale. LLMs can be loosely described as one answer to precisely this bottleneck: in effect, an implicit cliche library absorbed from an enormous body of code and code-related content. This is a metaphor, not a claim about what such models actually represent, and it comes at the price of having no explicit, inspectable representation of what a cliche is or what its roles are, and so of no

[5] One could argue that "training" and "loading" are not very different in concept as they both build up a knowledge base of one sort or another. However the loading of cliches and the training of a neural net are, of course, completely different forms of building up a knowledge base.

[6] Certainly an LLM harbours a tacit internal representation relevant to the structure of the code it is working on, but this is, as far as anyone can tell, quite unlike the explicit representations involved in the reasoning carried out by the historical programs studied here.

theory of programming knowledge that could be examined or tested.

Who does what? Simon's Heuristic Compiler was not designed as a practical programming assistant. It was a computational experiment intended to illuminate the processes involved in programming. KBEmacs, by contrast, was deliberately conceived as an assistant, leaving design decisions to the programmer while taking over routine details. Waters's argument for the apprentice was practical: full automation was not near, and a semiexpert assistant could be useful at once. Modern tools have traversed the same axis in a few years, from completing a line, to writing a function on request, to agents that plan, edit, and test across a code base with varying degrees of supervision [33, 9]. The question of where the programmer's decisions end and the machine's begin, central to the Programmer's Apprentice, is again a live design question.

How does the system know it is making progress? The Heuristic Compiler's executive is a loop that directly implements GPS's means-ends analysis: apply a compiler, test whether the routine has improved, stop if it reaches the goal or has not progressed. For Simon, this loop was not merely an engineering choice: means-ends analysis was Newell and Simon's theory of how people solve problems [15, 14], and the Heuristic Compiler was a test of whether that theory extended to programming. Modern coding agents run a loop of a similar shape, explicitly coded into the harnesses that make these tools work so well: reason about the task, act on the code, run the tests or the compiler, examine the difference between what happened and what was wanted, and repair what fails [35, 33, 25, 6].[7] The difference detector is now an executed test suite rather than a hand-coded comparison of states, and the operator is a learned model rather than a table of connections, but the architecture of means-ends iteration is recognizable. What survives, however, is the architecture, not the theory: such harnesses are built because the loop works, not as claims about how people program. The resemblance goes beyond the loop. A coding agent of this kind is not a purely neural system: the language model chooses what to do next and interprets the results, but the processes of compiling, executing, comparing outputs, tracking changes, and so on, are done by conventional symbolic programs (albeit not, for the most part, symbolic AIs). In the work reported here these were a Common Lisp compiler, an IPL-V interpreter, and KBEmacs's self-test. Although the heuristics have moved from a hand-written table of connections or cliche library, to a neural net, the operators and the difference tests are still ordinary programs.

Representation. As we have said, the two historical systems never treated code merely as text. The Heuristic Compiler builds IPL instructions out of description lists and prints them only at the end; KBEmacs reasons about cliches and plans and regenerates code after every command, taking pains to keep code and plan consistent when the user edits the code directly. Waters reports that "well over half of all the effort expended on KBEmacs went into implementing an analyzer and coder" [39, p. 55]. Today's assistants also only operate on text at the edges of the network, but their core technology is inscrutable.

6 Technical Next Steps

For the Heuristic Compiler, the undiscovered country is the English front end: the scanner, dictionary, and parser, and the routine that converts a parse into a functional description. Simon credited H. S. Kelly with "linguistic components for the compiler which are omitted from this report" [30, p. 506]; these routines may be them. Running them requires input sentences, which are not in the listing.

For KBEmacs, first, the compatibility layer was debugged against the one self-test sequence whose expected output was known, so the obvious next step is to run the other five sequences of

[7] It has not escaped our notice that the harness used for the reanimations reported here is among those so described.

the 1984 self-test. After that one could work on the Ada front end, which could allow Waters's main `UNIT-REPAIR-REPORT` demonstration to be reproduced. Another possibility would be to hook KBEmacs in an Emacs! Reproducing Zmacs would be difficult and useless, but it would be pleasing and amusing to be able to enter Control-Meta-Cokebottle into a modern unix-based Emacs and have it bring up KBEmacs!

7 Conclusion

Our reanimation of Simon's Heuristic Compiler and MIT's KBEmacs illuminates a history of AI and programming that is not simply a progression from primitive to increasingly sophisticated coding assistants. Simon's Heuristic Compiler investigated the nature of programming as problem solving. KBEmacs applied AI to assist human programmers. Both systems viewed the programming task as that of appropriately utilizing explicitly represented background knowledge. Although today's AI coding agents take on more and more of the work of programming themselves, these systems are not stages in a single technical lineage. Rather, they mark three different relationships between machines and programming: using methods now central to AI to understand programming, using AI to assist human programmers while also instantiating a theory of how human programmers program, and using AI to do more and more of the programming autonomously. By recovering and executing the original systems, the methodology of Executable Archaeology allows us to examine these relationships through the detailed study of the computational mechanisms themselves.

Like most early AI, both of the historical systems we have reanimated were, at least in part, attempts to understand how people program. Newell and Simon themselves long preferred the term "complex information processing" to "artificial intelligence." Simon's Heuristic Compiler treated programming as means-ends problem solving. The Programmer's Apprentice understood programming as combining familiar cliches. And note that the Heuristic Compiler also depended upon having a knowledge base of what amount to cliches. More importantly to the present effort, each was a theory of programming that could be run and tested, and we now can do so again.

Today's AI coding assistants program far better than either the Heuristic Compiler or KBEmacs, yet the LLMs at their core embody no comparable explicit theory of cliches or of the problem solving involved in programming. As we noted in Section 5, something of Simon's theory survives, as engineering, in the means-ends loop of the harnesses that drive these tools. What the harness leaves to the model, we venture, includes an implicit theory of problem solving, an implicit library of programming cliches, and an implicit skill in combining them, all distributed across the billions of weights that make up the LLM. That this knowledge is not stated in a form that can be examined is probably core to what makes these tools so flexible, but it also means that it cannot be reasoned about. An explicit theory can be. The Logic Theorist proved theorems by manipulating explicit logical expressions; the Heuristic Compiler carried the same kind of reasoning over to explicit descriptions of programs; and KBEmacs's Plan Calculus gave its cliches a formal semantics [19]. In each case the theory of programming was available both to the program, which reasoned with it, and to its builders, who could analyze it, test it, and in principle prove things about it. A theory distributed across the weights of an LLM can only be probed from the outside, by observing what the model does. One cannot prove anything about what one cannot represent.

8 Code Availability

The transcriptions, interpreters, compatibility layer, and Hacker's Guides to both programs are open source, at https://github.com/jeffshrager/ExecutableArchaeology and https:

//github.com/jeffshrager/kbemacs [23, 24]. The repositories contain the original source scans or provide links to them.

Author Contributions and Use of AI

J. Shrager conceived and directed the project, searched the Simon and Newell papers in the Carnegie Mellon University Archives with M. Golec, wrote the IPL-V interpreter on which the Heuristic Compiler runs, checked the results against the historical record, and edited the paper. K. M. Pitman, a member of the original KBEmacs team, provided the scans of the January 1985 KBEmacs listings and guidance on the Zetalisp compatibility layer required to run it. M. Golec worked with Shrager over an extended period in the Simon and Newell archives, and selected and scanned the Heuristic Compiler listing. All human authors read the paper and provided edits.

Most of the programming work described here was carried out by AI systems, under the authors' direction. Most of the transcription of the listings from the page images was done by ChatGPT (OpenAI) [4]. Claude (Anthropic), running as Claude Code [5] with the Claude Opus 5.5 model, corrected the transcriptions by running the code and checking the scans; adapted and extended Shrager's IPL-V interpreter; adapted g000001's Common Lisp ports of Spice Lisp Flavors and LetS [7], developed the Zetalisp compatibility layer for KBEmacs, and ran and debugged both programs until they reproduced their historical output. Claude also drafted this paper. The draft was then carefully checked by ChatGPT and by the authors, who verified its claims against the listings, the program runs, and the historical sources, and who take full responsibility for the final text.

These two reanimations required about 1,500 calls to the Claude Opus 5.5 model, about 1,600 tool invocations, and about six hours of active machine time (not counting ChatGPT's extraction of the KBEmacs listings from the scans).

Acknowledgments

We thank Emily Davis of Carnegie Mellon University, who supported Mia Golec's search of the Herbert A. Simon Papers in Carnegie Mellon University's Archives, resulting in locating the Heuristic Coder listing. We thank g000001 for the Common Lisp port of Spice Lisp Flavors and LetS (https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS) [7]. RAND Memorandum RM-3588-PR ([31]) was obtained from bitsavers (http://bitsavers.org/pdf/rand/ipl/). The Heuristic Coder listing is copyright CMU archives, and KBEmacs is copyright MIT. These are reproduced under the notice in those listings.

Finally, we acknowledge Herb Simon, H. S. Kelly, Richard Waters, Charles Rich, and the other members of the Programmer's Apprentice project, on whose work this paper rests.

References

[1] J. W. Backus et al., "The FORTRAN automatic coding system," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 188–198.

[2] R. M. Balzer, "A global view of automatic programming," in *Proc. 3rd Int. Joint Conf. Artif. Intell. (IJCAI)*, Stanford, CA, Aug. 1973, pp. 494–499.

[3] R. S. Boyer and J S. Moore, *A Computational Logic*. New York: Academic Press, 1979.

[4] OpenAI, "ChatGPT," 2026. [Online]. Available: https://chatgpt.com

[5] Anthropic, "Claude Code," 2026. [Online]. Available: https://www.anthropic.com/claude-code

[6] Anthropic, "How Claude Code works," Claude Code documentation, 2026. [Online]. Available: https://code.claude.com/docs/en/how-claude-code-works

[7] g000001, "spice-flavors" and "LetS," GitHub. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS

[8] C. Green, "Application of theorem proving to problem solving," in *Proc. 1st Int. Joint Conf. Artif. Intell.*, 1969, pp. 219–240.

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[10] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, "ELIZA reanimated: Restoring the mother of all chatbots to one of the world's first time-sharing systems," *IEEE Ann. Hist. Comput.*, vol. 47, no. 2, pp. 68–76, Apr.-Jun. 2025.

[11] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, no. 3, pp. 151–165, Mar. 1971.

[12] D. Moews and J. Shrager, "Executable archaeology: Reanimating the Logic Theorist," *IEEE Ann. Hist. Comput.*, submitted, 2026.

[13] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem-solving program," in *Proc. Int. Conf. Inform. Process.*, Paris: UNESCO, 1960, pp. 256–264.

[14] A. Newell and H. A. Simon, *Human Problem Solving*. Englewood Cliffs, NJ: Prentice-Hall, 1972.

[15] A. Newell, J. C. Shaw, and H. A. Simon, "Elements of a theory of human problem solving," *Psychol. Rev.*, vol. 65, no. 3, pp. 151–166, 1958.

[16] A. Newell, J. C. Shaw, and H. A. Simon, "Empirical explorations of the logic theory machine: A case study in heuristic," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.

[17] K. M. Pitman, "netsettlement: kbe," GitHub (scans of *Knowledge-Based Editor*, four-volume listing, Jan. 1985). Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/netsettler/netsettlement/tree/main/kbe

[18] C. Rich and H. E. Shrobe, "Initial report on a LISP programmer's apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-354, Dec. 1976. (Also *IEEE Trans. Softw. Eng.*, vol. SE-4, no. 6, pp. 456–467, Nov. 1978.)

[19] C. Rich, "A formal representation for plans in the Programmer's Apprentice," in *Proc. 7th Int. Joint Conf. Artif. Intell. (IJCAI)*, Vancouver, BC, Canada, 1981, pp. 1044–1052.

[20] C. Rich and R. C. Waters, "The Programmer's Apprentice project: A research overview," MIT AI Lab., Cambridge, MA, USA, AI Memo 1004, Nov. 1987.

[21] C. Rich and R. C. Waters, "Automatic programming: Myths and prospects," *Computer*, vol. 21, no. 8, pp. 40–51, Aug. 1988.

[22] C. Rich and R. C. Waters, *The Programmer's Apprentice*. New York, NY, USA: ACM Press, and Reading, MA, USA: Addison-Wesley, 1990.

[23] J. Shrager, "Executable Archaeology," GitHub (HeuristicCoder, IPL-V, and LogicTheorist directories). Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/ExecutableArchaeology

[24] J. Shrager, "kbemacs: MIT's KBEmacs from the January 1985 listings, running under SBCL," GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/kbemacs

[25] E. Schluntz and B. Zhang, "Building effective agents," Anthropic, Dec. 19, 2024. [Online]. Available: https://www.anthropic.com/research/building-effective-agents

[26] J. Shrager, "Executable Archaeology: Reanimating the Logic Theorist from its IPL-V source," Mar. 13, 2026, *arXiv:2603.13514v1*. [Online]. Available: https://arxiv.org/abs/2603.13514

[27] J. Shrager. "An IPL-V interpreter in Common Lisp that runs the original Logic Theorist," GitHub (archived; superseded by [23]). [Online]. Available: https://github.com/jeffshrager/IPL-V

[28] H. A. Simon, "Experiments with a heuristic compiler," RAND Corp., Santa Monica, CA, USA, Paper P-2349, Jun. 30, 1961.

[29] H. A. Simon, "Heuristic Coder 7/16/61," IPL-V program listing, Herbert A. Simon Papers, Box 14, Folder FF965, Carnegie Mellon Univ. Archives, Pittsburgh, PA, USA.

[30] H. A. Simon, "Experiments with a heuristic compiler," *J. ACM*, vol. 10, no. 4, pp. 493–506, Oct. 1963.

[31] H. A. Simon, "The heuristic compiler," RAND Corp., Santa Monica, CA, USA, Memorandum RM-3588-PR, May 1963. [Online]. Available: https://bitsavers.trailing-edge.com/pdf/rand/ipl/RM-3588-PR_The_Heuristic_Compiler_May63.pdf

[32] E. Soloway and K. Ehrlich, "Empirical studies of programming knowledge," *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 5, pp. 595–609, Sep. 1984.

[33] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[34] G. M. Weinberg, *The Psychology of Computer Programming*. New York, NY, USA: Van Nostrand Reinhold, 1971.

[35] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[36] R. C. Waters, "Automatic analysis of the logical structure of programs," Ph.D. dissertation, MIT, Cambridge, MA, USA, 1978. (MIT AI Lab. Tech. Rep. AI-TR-492.)

[37] R. C. Waters, "KBEmacs: A step toward the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-753, May 1985.

[38] R. C. Waters, "The Programmer's Apprentice: A session with KBEmacs," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 11, pp. 1296–1320, Nov. 1985.

[39] R. C. Waters, "KBEmacs: Where's the AI?" *AI Mag.*, vol. 7, no. 1, pp. 47–56, Spring 1986.

About the Authors

Jeff Shrager is a cognitive scientist, computer scientist, and molecular biologist. In addition to his work on Executable Archaeology, he is developing CogLean, a Lean-based library of formal components that can be used to prove claims about cognitive models. He is currently Chief Scientist of Bennu Climate, Inc.
Kent M. Pitman {Kent M. Pitman to provide}
Mia Golec {Mia Golec to provide}